

格点 QCD 中胶子 PDF 计算的 物理公式与 GPU 代码实现

——donghx 代码详细分析——

张欣*

基于 `/root/lattice-pdf/examples/donghx/` 中全部代码及
`/root/lattice-pdf/docs/` 中的参考文献

2026 年 7 月 27 日

目录

1	引言与物理背景	3
1.1	LaMET 理论框架	3
1.2	胶子 PDF 的矩阵元	3
1.3	非连通图与 OPE 分解	4
2	规范场读入与数据组织	4
2.1	ILDG 二进制格式	4
2.2	参数传递方式	5
3	场强张量 $F_{\mu\nu}$ 的格点构造	6
3.1	物理定义：从 Plaquette 到 $F_{\mu\nu}$	6
3.2	Clover（四叶草）Plaquette 改进	6
3.3	代码实现：plaquette_clover_new	8
3.4	代码实现：plaquette_clover_all_new	9
3.5	对偶 Plaquette： $\tilde{P}_{\mu\nu}$ 的构造	10

*中国科学院近代物理研究所 (IMP, CAS)

4	非定域胶子 OPE 算符的构造	10
4.1	带 Wilson 线的非定域算符	10
4.2	完整 OPE 算符的代码实现	11
4.3	OPE 算符的各种变体	13
4.4	OPE 计算中的 MPI 并行	13
5	质子 2pt 关联函数: Distillation + 动量涂抹	15
5.1	Distillation 方法基础	15
5.2	本征矢量 (Eigenvector) 读入	15
5.3	Perambulator 读入与结构	16
5.4	VVV: 三夸克重子顶点	18
5.5	Wick 收缩与 2pt 关联函数	20
5.6	宇称投影	21
6	Gamma 矩阵: DeGrand-Rossi (DR) 基	23
6.1	DR 基的显式表示	23
6.2	代码实现	24
6.3	插值算符 (Interpolator) 的构造	24
7	GPU 计算策略与数据管理	26
7.1	CuPy 后端	26
7.2	内存管理策略	26
7.3	opt_einsum 加速	27
7.4	DCU (AMD/Hygon) 后端	27
7.5	文件 I/O 与数据格式	28
8	完整计算流程与数据流	28
8.1	端到端流程	28
8.2	各步骤的代码对应	30
8.3	典型运行命令	30
9	总结	31

1 引言与物理背景

1.1 LaMET 理论框架

大动量有效理论 (Large Momentum Effective Theory, LaMET) 是连接格点 QCD 欧几里得关联函数与光锥 PDF 的桥梁 [1, 2, 3]。其核心思想是：在格点上计算一个大动量 P_z 下的准 PDF (quasi-PDF) $\tilde{g}(x, P_z)$ ，然后通过微扰匹配核 (perturbative matching kernel) 将其与光锥 PDF $g(x, \mu)$ 联系起来：

LaMET 因子化公式

$$\tilde{g}(x, P_z) = \int_0^1 \frac{dy}{|y|} C\left(\frac{x}{y}, \frac{\mu}{P_z}\right) g(y, \mu) + \mathcal{O}\left(\frac{\Lambda_{\text{QCD}}^2}{P_z^2}, \frac{\Lambda_{\text{QCD}}^2}{x^2 P_z^2}\right) \quad (1)$$

其中 $C(x/y, \mu/P_z)$ 是微扰匹配核，在 NLO 精度下已知。准 PDF $\tilde{g}(x, P_z)$ 由格点上的非定域关联函数通过 Fourier 变换得到：

准 PDF 的 Fourier 变换定义

$$\tilde{g}(x, P_z) = \int_{-\infty}^{\infty} \frac{dz}{2\pi} e^{ixP_z z} \tilde{h}(z, P_z) \quad (2)$$

其中 $\tilde{h}(z, P_z)$ 是坐标空间中的非定域关联函数 (IOG 关联函数)，对于胶子 PDF，它由场强张量 $F_{\mu\nu}$ 构造。

1.2 胶子 PDF 的矩阵元

对于质子中的非极化胶子 PDF，我们要计算的矩阵元是 [4, 5]：

胶子 PDF 矩阵元—内部 note Eq(20)

$$\tilde{h}(z, P_z) = \frac{\langle P | \mathcal{O}_{\mu\nu;\rho\sigma}(z) | P \rangle}{\langle P | P \rangle} \quad (3)$$

其中非定域胶子算符为：

$$\mathcal{O}_{\mu\nu;\rho\sigma}(z) = F_{\mu\nu}(0) U(0 \rightarrow z\hat{n}) F_{\rho\sigma}(z\hat{n}) U(z\hat{n} \rightarrow 0) \quad (4)$$

这里 $F_{\mu\nu}$ 是胶子场强张量， $U(0 \rightarrow z\hat{n})$ 是沿方向 $\hat{n}$ 的 Wilson 线 (规范连接)。在非极化情况下，我们需要对 Lorentz 指标 $(\mu\nu)$ 和 $(\rho\sigma)$ 求和。在格点上， $F_{\mu\nu}$ 由四叶草 (Clover) plaquette 构造，Wilson 线由规范链路上的 SU(3) 矩阵乘积给出。

1.3 非连通图与 OPE 分解

胶子 PDF 涉及非连通图 (disconnected diagram)。三点关联函数可以分解为质子 2pt 关联函数与 OPE (Operator Product Expansion) 部分 [4]:

三点关联函数的 OPE 分解—内部 note Eq(25)

$$C_3(t, \tau; \vec{P}) = \sum_{\vec{x}} e^{-i\vec{P} \cdot \vec{x}} \langle \chi(\vec{x}, t) \mathcal{O}_{\text{gluon}}(z, \tau) \bar{\chi}(\vec{0}, 0) \rangle \approx C_2(t; \vec{P}) \cdot \mathcal{O}_{\text{OPE}}(z, \tau) \quad (5)$$

因此, 整个计算分为两个独立的部分:

- (1) **质子 2pt 关联函数** $C_2(t; \vec{P})$: 使用动量涂抹 (momentum smearing) 的蒸馏 (distillation) 方法计算
- (2) **OPE 胶子算符** $\mathcal{O}_{\text{OPE}}(z, \tau)$: 由场强张量和 Wilson 线构造的非定域算符

2 规范场读入与数据组织

2.1 ILDG 二进制格式

格点规范组态以 ILDG (International Lattice Data Grid) 二进制格式存储。在 donghx 的代码中, 规范场按以下方式读取 (参考 Calc_pla.py 和 Calc_ope_unpol.py):

规范场读入—Calc_pla.py:29-41

```
1 f = open("%s" % conf_file, "rb")
2 gauge = np.fromfile(f, dtype=">f8")
3 gauge = np.array(gauge)
4
5 gauge = gauge.reshape(Nt, Nx, Nx, Nx, 4, 3, 3, 2)
6 gauge = gauge[..., 0] + gauge[..., 1] * 1j
7 f.close()
```

规范场张量结构

读取并 reshape 后, 规范链路的张量形状为:

$$U_{\mu}(x) \longleftrightarrow \text{gauge}[t, z, y, x, \text{dir}, \text{color_row}, \text{color_col}] \quad (6)$$

即 [Nt, Nz, Ny, Nx, 4, 3, 3] 的复数张量。

空间指标约定 (donghx 代码):

- 轴 0: t (时间方向), 轴 1: z, 轴 2: y, 轴 3: x
- 方向 μ : 0=x, 1=y, 2=z, 3=t
- 颜色指标 (0,1,2): SU(3) 基本表示

注意: 这与 zhangxin 代码中的约定不同 (后者使用 [color, color, dir, x, y, z, t]), 在跨代码使用时需要 transpose 操作。

2.2 参数传递方式

donghx 的代码使用 stdin 重定向方式传递参数 (而非 argparse), 参数格式为空格分隔的键值对:

参数传递—Calc_ope_unpol.py:20-37

```

1 infile = fileinput.input()
2 for line in infile:
3     tmp = line.split()
4     if tmp[0] == "Nt":
5         Nt = int(tmp[1])
6     if tmp[0] == "Nx":
7         Nx = int(tmp[1])
8     if tmp[0] == "delta_z":
9         delta_z = int(tmp[1])
10    if tmp[0] == "conf_id":
11        conf_id = tmp[1]
12    if tmp[0] == "conf_file":
13        conf_file = tmp[1]
14    if tmp[0] == "output_dir":
15        output_dir = tmp[1]
```

运行时通过 shell 重定向传入参数文件:

```
1 python Calc_ope_unpol.py < params.txt
```

典型的 params.txt 内容:

```
Nt 64
Nx 32
delta_z 15
conf_id 20000
conf_file /path/to/config.dat
link_dir z
output_dir /path/to/output/
```

3 场强张量 $F_{\mu\nu}$ 的格点构造

3.1 物理定义：从 Plaquette 到 $F_{\mu\nu}$

在连续极限下，规范场强张量 $F_{\mu\nu}$ 由 Wilson loop 的 1×1 plaquette 给出：

Plaquette 与场强张量的关系

1×1 plaquette 在点 x 处的定义为：

$$P_{\mu\nu}(x) = U_\mu(x)U_\nu(x + \hat{\mu})U_\mu^\dagger(x + \hat{\nu})U_\nu^\dagger(x) \quad (7)$$

在连续极限 $a \rightarrow 0$ 下：

$$P_{\mu\nu}(x) = \exp(ia^2gF_{\mu\nu}(x) + \mathcal{O}(a^3)) \approx \mathbb{1} + ia^2gF_{\mu\nu}(x) \quad (8)$$

因此场强张量可以从 plaquette 的反厄米无迹部分提取：

$$F_{\mu\nu}(x) = \frac{1}{2ia^2g}(P_{\mu\nu}(x) - P_{\mu\nu}^\dagger(x)) - \frac{1}{3}\text{Tr}[\cdots] \quad (9)$$

3.2 Clover (四叶草) Plaquette 改进

为提高精度，实际计算中使用 clover (四叶草) 改进的 plaquette。Clover 构型在点 x 周围使用四个 plaquette 的对称组合，将离散化误差从 $\mathcal{O}(a)$ 降低到 $\mathcal{O}(a^2)$ ：

Clover 改进的场强张量

$$F_{\mu\nu}^{\text{clover}}(x) = -\frac{i}{8a^2g} \left[Q_{\mu\nu}(x) - Q_{\mu\nu}^\dagger(x) \right]_{\text{traceless}} \quad (10)$$

其中 $Q_{\mu\nu}$ 是四个定向 plaquette 的和：

$$Q_{\mu\nu}(x) = P_{\mu\nu}(x) + P_{\nu,-\mu}(x) + P_{-\mu,-\nu}(x) + P_{-\nu,\mu}(x) \quad (11)$$

四个 plaquette 从同一点 x 出发，分别沿 (μ, ν) , $(\nu, -\mu)$, $(-\mu, -\nu)$, $(-\nu, \mu)$ 方向：

- $P_{\mu\nu}$: 右上方 1×1 plaquette
- $P_{\nu,-\mu}$: 左上方，从 x 沿 $+\nu$ 再沿 $-\mu$
- $P_{-\mu,-\nu}$: 左下方，从 x 沿 $-\mu$ 再沿 $-\nu$
- $P_{-\nu,\mu}$: 右下方，从 x 沿 $-\nu$ 再沿 $+\mu$

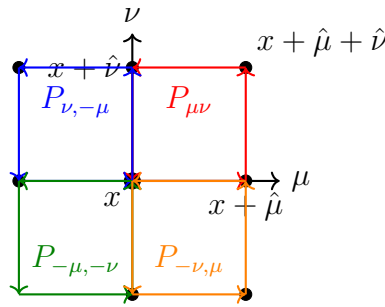


图: Clover (四叶草) plaquette 构型: 从点 x 出发的四个定向 plaquette

3.3 代码实现: `plaquette_clover_new`

单个 Clover Plaquette 计算—Operator.py:77-161

```

1 def plaquette_clover_new(gauge, mu, nu):
2     # 四个起始点: 通过roll操作平移整格
3     gauge_rightup = gauge
4     gauge_leftup = np.roll(gauge, 1, 3 - mu)
5     gauge_rightdown = np.roll(gauge, 1, 3 - nu)
6     gauge_leftdown = np.roll(gauge_leftup, 1, 3 - nu)
7
8     # ---  $P_{\{\mu, \nu\}}$  ( $pla\_rightup$ ) ---
9     pla_rightup = np.einsum(
10         "tzyxab,tzyxbc->tzyxac",
11         gauge_rightup[:, :, :, :, mu, :, :],
12         np.roll(gauge_rightup, -1, 3 - mu)[ :, :, :, :, nu, :, :],
13     )
14     pla_rightup = np.einsum(
15         "tzyxab,tzyxcb->tzyxac",
16         pla_rightup,
17         np.roll(gauge_rightup, -1, 3 - nu)[ :, :, :, :, mu, :, :].conj(),
18     )
19     pla_rightup = np.einsum(
20         "tzyxab,tzyxcb->tzyxac",
21         pla_rightup,
22         gauge_rightup[:, :, :, :, nu, :, :].conj(),
23     )
24     # ... (类似地计算  $pla\_leftup$ ,  $pla\_leftdown$ ,  $pla\_rightdown$ )
25
26     ans = (
27         pla_rightup - pla_rightup.conj().transpose(0,1,2,3,5,4)
28         + pla_leftup - pla_leftup.conj().transpose(0,1,2,3,5,4)
29         + pla_leftdown - pla_leftdown.conj().transpose
30             (0,1,2,3,5,4)
31         + pla_rightdown - pla_rightdown.conj().transpose
32             (0,1,2,3,5,4)
33     )
34     return -1j * ans / 8.0

```


代码-物理对应: Clover Plaquette

代码中的每个 einsum 对应一个 SU(3) 矩阵乘法 (沿封闭路径的颜色指标缩并):

- `einsum("tzyxab,tzyxbc->tzyxac", A, B):  $A_{ab}B_{bc} = (AB)_{ac}$`
- `einsum("tzyxab,tzyxcb->tzyxac", A, B):  $A_{ab}B_{cb} = A_{ab}(B^\dagger)_{bc} = (AB^\dagger)_{ac}$`
- `np.roll(gauge, -1, 3-mu):` 空间平移 $x \rightarrow x + \hat{\mu}$ (将格点指标沿 μ 方向偏移)
- `.conj():` 厄米共轭 $U^\dagger$
- 最终返回的 `-1j * ans / 8.0:` $-\frac{i}{8}Q_{\mu\nu}^{\text{anti-hermitian}}$

四个 plaquette 的遍历顺序利用了 $\varepsilon^{\mu\nu\rho\sigma}$ 的反对称性: $P_{\mu\nu}$ 与 $P_{\nu\mu}$ 给出相反的贡献, 因此最终结果自然满足 $F_{\mu\nu} = -F_{\nu\mu}$ 。

3.4 代码实现: plaquette_clover_all_new

该函数遍历所有 (μ, ν) 组合生成完整的 4×4 plaquette 张量:

全部 plaquette 生成—Operator.py:164-174

```

1 def plaquette_clover_all_new(gauge, Nt, Nx):
2     pla = np.zeros((4, 4, Nt, Nx, Nx, Nx, 3, 3), dtype=complex)
3     Temp_zeros = np.zeros((Nt, Nx, Nx, Nx, 3, 3), dtype=complex)
4     for _mu in range(4):
5         for _nu in range(4):
6             if _mu == _nu:
7                 pla[_mu, _nu] = Temp_zeros # 对角元为0 (反对称性)
8             else:
9                 pla[_mu, _nu] = plaquette_clover_new(gauge, _mu, _nu)
10    return pla

```

输出张量形状为 `[4, 4, Nt, Nz, Ny, Nx, 3, 3]`:

- 前两个维度: (μ, ν) 空间方向指标
- 中间四个维度: 时空坐标 (t, z, y, x)
- 最后两个维度: SU(3) 颜色矩阵

3.5 对偶 Plaquette: $\tilde{P}_{\mu\nu}$ 的构造

胶子 OPE 算符中需要使用对偶场强张量 $\tilde{F}_{\mu\nu} = \frac{1}{2}\varepsilon_{\mu\nu\rho\sigma}F_{\rho\sigma}$ 。代码中通过 Tensor4 (即 $\frac{1}{2}\varepsilon_{\mu\nu\rho\sigma}$) 将对偶变换实现为张量缩并:

对偶 Plaquette —Operator.py:177-184

```

1 Tensor4 = np.zeros((4, 4, 4, 4), dtype=complex)
2 # ... 构造 epsilon 张量: Tensor4[i,j,k,l] = 1/2 * epsilon_{i,j,k,l}
3 # 当 (i,j,k,l) 为偶排列时取 +1/2, 奇排列时取 -1/2
4
5 def plaquette_clover_all_tilde(pla_all, Nt, Nx):
6     pla_tilde = contract(
7         "opmn,mntzyxab->optzyxab",
8         Tensor4, # (1/2) * epsilon_{mu,nu,rho,sigma}
9         pla_all, # F_{rho,sigma}(x)
10    )
11    return pla_tilde

```

对偶变换的数学表达

$$\tilde{P}_{\mu\nu}(x) = \frac{1}{2}\varepsilon_{\mu\nu\rho\sigma} P_{\rho\sigma}(x) \quad (12)$$

在胶子 OPE 中, 非极化矩阵元需要 $F_{\mu\nu}$ 与 $\tilde{F}_{\mu\nu}$ 的组合:

$$\mathcal{O}_{\text{unpol}} \sim F_{\mu\nu}(0) U(0 \rightarrow z) \tilde{F}_{\mu\nu}(z) U(z \rightarrow 0) \quad (13)$$

(注意: 实际的非极化算符根据 note Eq(25) 有不同的 Lorentz 结构组合。)

4 非定域胶子 OPE 算符的构造

4.1 带 Wilson 线的非定域算符

OPE 算符的一般形式 [4, 6] 是:

非定域胶子算符

$$\mathcal{O}_{\mu\nu;\rho\sigma}(z\hat{n}) = F_{\mu\nu}(0) \underbrace{U(0 \rightarrow z\hat{n})}_{\text{Wilson 线}} F_{\rho\sigma}(z\hat{n}) \underbrace{U(z\hat{n} \rightarrow 0)}_{\text{返回的 Wilson 线}} \quad (14)$$

在格点上，Wilson 线是沿路径的规范链路乘积：

$$U(0 \rightarrow z\hat{n}) = \prod_{k=0}^{z/a-1} U_n(ka\hat{n}) \quad (15)$$

返回路径由厄米共轭给出： $U(z\hat{n} \rightarrow 0) = U(0 \rightarrow z\hat{n})^\dagger$

4.2 完整 OPE 算符的代码实现

代码提供了多种 OPE 算符实现,对应不同的空间结构和边界条件。以 `operators_new_z0_mu2` (最完整的带完整 Wilson 线的实现) 为例：

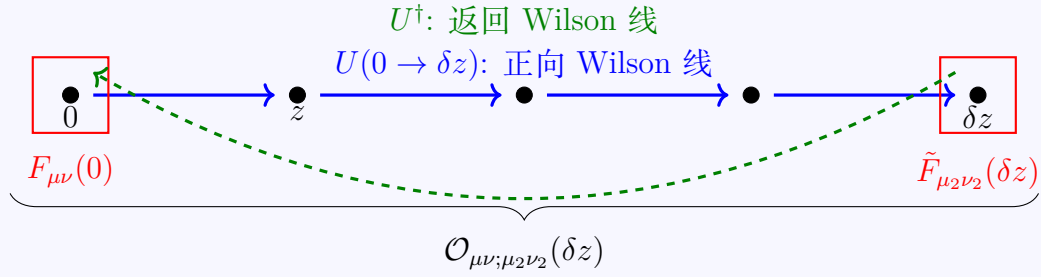
完整 OPE 算符—Operator.py:339-368

```

1 def operators_new_z0_mu2(gauge, zdir, pla, pla_tilde,
2                           delta_z, mu, nu, mu2, nu2, Nt):
3     op = np.zeros(Nt, dtype=complex)
4     z_dir = zdir # Wilson线方向: 0=x, 1=y, 2=z
5
6     # Step 1: 将F_{mu,nu}平移到位置 z-dz
7     ope = np.roll(pla[mu, nu], -delta_z, 3 - z_dir)
8
9     # Step 2: 构造返回的Wilson线 U(dz->0) = 从dz沿-z方向回来
10    for _dz in range(delta_z):
11        ope = np.einsum(
12            "tzyxab,tzyxcb->tzyxac",
13            ope,
14            np.roll(gauge, -(delta_z - 1 - _dz), 3 - z_dir)[
15                :, :, :, :, z_dir, :, :
16            ].conj(),
17        )
18
19    # Step 3: 乘上位置0处的对偶plaquette tilde{F}_{mu2,nu2}
20    ope = np.einsum(
21        "tzyxab,tzyxbc->tzyxac",
22        ope,
23        pla_tilde[mu2, nu2],
24    )
25
26    # Step 4: 构造向前的Wilson线 U(0->dz)
27    for _dz in range(delta_z):
28        ope = np.einsum(
29            "tzyxab,tzyxbc->tzyxac",
30            ope,
31            np.roll(gauge, -_dz, 3 - z_dir)[: , :, :, :, z_dir,
32                :, :],
33        )
34
35    # Step 5: 对颜色空间取迹, 对空间求和
36    ans = np.trace(op, axis1=4, axis2=5)
37    op = np.sum(ans, axis=(1, 2, 3))
38
39    return op # shape: [Nt]

```

代码-物理对应：OPE 算符构造



图：非定域 OPE 算符的格点结构示意图

代码的五个步骤对应于：

Step 1: **位置偏移**：用 `np.roll` 将 $F_{\mu\nu}$ 从位置 0 平移到位置 δz

Step 2: **返回 Wilson 线**：通过 `delta_z` 步的厄米共轭链路乘积，从 δz 走回到 0。
 每一步收缩对应：

$$\text{o pe}_{ac} \leftarrow \text{o pe}_{ab} \cdot U_n^\dagger(b\hat{n})_{bc} \quad (16)$$

Step 3: **插入 $\tilde{F}$** ：在位置 0 处乘以对偶 plaquette $\tilde{P}_{\mu_2\nu_2}(0)$

Step 4: **正向 Wilson 线**：从 0 走向 δz ，每一步：

$$\text{o pe}_{ac} \leftarrow \text{o pe}_{ab} \cdot U_n(a\hat{n})_{bc} \quad (17)$$

Step 5: **颜色求迹 + 空间求和**：得到时间片 t 上的 OPE 算符值

4.3 OPE 算符的各种变体

`donghx` 代码中实现了多种 OPE 算符变体，适应不同的物理需求：

4.4 OPE 计算中的 MPI 并行

在 `Calc_ope_unpol.py` 和 `Calc_ope_helicity.py` 中，使用 MPI 将不同的 (μ, ν) 分量分配给不同的 `rank`：

表 1: Operator.py 中的 OPE 算符函数汇总

函数名	Wilson 线结构	输出形状
operators_FF_z0	无 Wilson 线, $\tilde{F}$ 在位置 z	[Nt]
operators_new	完整双向 Wilson 线	[Nt, Nz, Ny, Nx, 3, 3]
operators_new_shift	完整双向 Wilson 线, 平移一个格距	[Nt, Nz, Ny, Nx, 3, 3]
operators_new_z0	$\tilde{F}$ 在 0 位置, 正向再返回	[Nt]
operators_new_z0_mz	同上但负 z 方向	[Nt]
operators_new_z0_mu2	支持任意 (μ_2, ν_2) 指标	[Nt]
operators_new_z0_mz_mu2	负 z + 任意指标	[Nt]
operators_new_z0_mu2_xy	保留 x, y 依赖	[Nt, Nx, Nx]
operators_new_z0_mz_mu2_xy	负 z + x, y 保留	[Nt, Nx, Nx]

MPI 并行分量分配—Calc_ope_unpol.py:76-93

```

1 if rank == 0:
2     _mu = 3;    _mu2 = 3
3     _nu = (zdir + 1) % 3;    _nu2 = (zdir + 1) % 3
4     # 计算:  $F_{\{3, zdir+1\}} \cdot U \cdot \tilde{F}_{\{3, zdir+1\}} \cdot U^\dagger$ 
5
6 if rank == 1:
7     _mu = 3;    _mu2 = 3
8     _nu = (zdir + 2) % 3;    _nu2 = (zdir + 2) % 3
9     # 计算:  $F_{\{3, zdir+2\}} \cdot U \cdot \tilde{F}_{\{3, zdir+2\}} \cdot U^\dagger$ 
10
11 if rank == 2:
12     _mu = (zdir + 1) % 3;    _mu2 = (zdir + 1) % 3
13     _nu = (zdir + 2) % 3;    _nu2 = (zdir + 2) % 3
14     # 计算:  $F_{\{zdir+1, zdir+2\}} \cdot U \cdot \tilde{F}_{\{zdir+1, zdir+2\}} \cdot U^\dagger$ 

```

三个 Lorentz 分量的物理意义

对于 Wilson 线沿 z 方向的情况 (zdir=2), 三个分量分别对应:

- **Rank 0:** $F_{3,0}$ —时间-空间分量 F_{tx} (电场型)
- **Rank 1:** $F_{3,1}$ —时间-空间分量 F_{ty} (电场型)
- **Rank 2:** $F_{0,1}$ —空间-空间分量 F_{xy} (磁场型)

对于**非极化** (unpolarized) 胶子 PDF (Calc_ope_unpol.py), 不需要对偶变换 (直接使用 pla 而非 pla_tilde), 意味着只计算 $F_{\mu\nu} \cdot U \cdot F_{\mu\nu} \cdot U^\dagger$ 型算符。

对于**螺旋度** (helicity) 胶子 PDF (Calc_ope_helicity.py), 使用 pla_tilde (对偶 plaquette), 即计算 $F_{\mu\nu} \cdot U \cdot \tilde{F}_{\mu\nu} \cdot U^\dagger$ 型算符, 其中 $\tilde{F}$ 对应于 $\varepsilon^{\mu\nu\rho\sigma} F_{\rho\sigma}$ 。

5 质子 2pt 关联函数: Distillation + 动量涂抹

5.1 Distillation 方法基础

Distillation 方法 [7, 8] 是一种计算强子关联函数的数值技术。它利用了拉普拉斯算符的低模本征矢量空间作为夸克场的涂抹 (-smearing) 基:

Distillation 的夸克场展开

在格点上, 夸克传播子将源点和汇点的夸克场通过本征矢量基展开:

$$S(x, y)_{\alpha\beta}^{ab} \approx \sum_{k,l=1}^{N_{\text{ev}}} v^{(k)}(x)_\alpha^a \tau^{(k,l)}(t_x, t_y)_{\alpha\beta} v^{(l)\dagger}(y)_\beta^b \quad (18)$$

其中:

- $v^{(k)}(x)$: 第 k 个拉普拉斯本征矢量 (在空间格点 x 上, 带颜色和自旋指标)
- $\tau^{(k,l)}(t_x, t_y)$: perambulator (传播子在 distillation 基中的矩阵元)
- N_{ev} : 截断的本征矢量数目 (典型值 100-200)

5.2 本征矢量 (Eigenvector) 读入

本征矢量是 3D 拉普拉斯算符 ∇^2 的低能本征态, 存储在二进制文件中:

本征矢量读入—2pt_proton_Cg5gmu_L32x64_mom2_xdir_gpu.py:172-186

```

1 def readin_eigvecs(eig_dir, t, Nev1, conf_id, Nx):
2     filename = f"%s/eigvecs_t%03d_%s" % (eig_dir, t, conf_id)
3     with open(filename, "rb") as f:
4         file_size = os.path.getsize(filename)
5         element_size = 8
6         Nev = int(file_size / element_size / (Nx*Nx*Nx * 3 * 2))
7         data = np.fromfile(f, dtype="f8").reshape(Nev, Nx, Nx,
8             Nx, 3, 2)
9         Eigvec = data[..., 0] + 1j * data[..., 1]
10        Eigvec = Eigvec[0:Nev1]
11        eigvecs_cupy = cp.asarray(Eigvec)
12        eigvecs_cupy = eigvecs_cupy.reshape(Nev, Nx*Nx*Nx, 3)
13        # 动量涂抹：乘上平面波相位因子
14        eigvecs_mom2 = contract("vxa,x->vxa", eigvecs_cupy,
15            phase_factor)
16    return eigvecs_mom2

```

动量涂抹 (Momentum Smearing)

动量涂抹通过乘上平面波相位因子实现 [9, 10]:

$$v_{\text{mom}}^{(k)}(x) = e^{-i\vec{\zeta} \cdot \vec{x}} v^{(k)}(x) \quad (19)$$

其中涂抹动量 $\vec{\zeta}$ 控制动量的注入量。例如 `momsmeas_phase=-3` 意味着 $\vec{\zeta} = (0, 0, -3) \times 2\pi/L$ 。

物理动机：直接在大动量 P_z 下计算强子 2pt 关联函数会因信噪比下降而困难。动量涂抹通过给本征矢量注入一个平面波相位，提高了与高动量态的 overlap，从而在不显著增加统计噪声的情况下提高信号质量。

5.3 Perambulator 读入与结构

Perambulator $\tau(t_{\text{sink}}, t_{\text{source}})_{\alpha\beta}^{(k,l)}$ 是夸克传播子在 distillation 基中的表示：

Perambulator 读入—2pt_proton_Cg5gm_u_L32x64_mom2_xdir_gpu.py:236

```

1 def readin_peram(peram_dir, conf_id, Nt, Nev1, t_source):
2     # 读入四个自旋分量的 perambulator 文件
3     f = open("%s/perams.%s.0.%i" % (peram_dir, conf_id, t_source
4         ), "rb")
5     peram = np.fromfile(f, dtype="f8")
6     f.close()
7     for d_source in range(1, 4):
8         f = open("%s/perams.%s.%i.%i" % (peram_dir, conf_id,
9             d_source, t_source), "rb")
10        temp = np.fromfile(f, dtype="f8")
11        peram = np.append(peram, temp)
12        f.close()
13
14    peram_size = peram.size
15    Nev = int(np.sqrt(peram_size / (4 * 4 * Nt * 2)))
16    # 重塑: d_source, t_sink, ev_source, d_sink, ev_sink,
17    complex
18    peram = peram.reshape(4, Nt, Nev, 4, Nev, 2)
19    # 转置: t_sink, d_sink, d_source, ev_sink, ev_source,
20    complex
21    peram = peram.transpose(1, 3, 0, 4, 2, 5)
22    peram = peram[..., 0] + peram[..., 1] * 1j
23    peram = peram[:, :, :, 0:Nev1, 0:Nev1]
24    peram_cupy = cp.array(peram)
25    return peram_cupy

```

Perambulator 张量结构

Perambulator 的张量形状:

$$\tau(t_{\text{sink}}, t_{\text{source}})_{d_{\text{sink}}, d_{\text{source}}}^{(k, l)} \longleftrightarrow [\text{Nt}, 4, 4, \text{Nev1}, \text{Nev1}] \quad (20)$$

各维度的物理含义:

- [Nt]: t_{sink} 时间汇 (对于固定的 t_{source})
- [4]: d_{sink} Dirac 自旋指标 (汇)
- [4]: d_{source} Dirac 自旋指标 (源)
- [Nev1]: l 汇本征矢量指标
- [Nev1]: k 源本征矢量指标

注: t_{source} 通过外层循环遍历。在 GPU 版本中每个 t_{source} 按需读入, 在 CPU 版本 (readin_peram_all_cpu) 中一次性读入所有 t_{source} 。

5.4 VVV: 三夸克重子顶点

VVV (Vector-Vector-Vector) 是三夸克重子 (质子/中子) 在 distillation 基中的组态因子:

VVV 计算—2pt_proton_Cg5gmu_L32x64_mom2_xdir_gpu.py:244-318

```

1 def VVV_Calc_cupy(Mom):
2     VVV_sink = cp.zeros((Nt, Nev1, Nev1, Nev1), dtype=complex)
3     for t in range(0, Nt):
4         eigvecs_cupy = readin_eigvecs(eig_dir, t, Nev1, conf_id,
5                                     Nx)
6         phase_factor_cupy = phase_calc(Mom)
7         for xi in range(0, Nx): # 分块处理以避免GPU内存溢出
8             VVV_sink[t] = (
9                 VVV_sink[t]
10                + contract("x,ax,bx,cx->abc",
11                           phase_factor_cupy[xi*Nx**2:(xi+1)*Nx**2],
12                           eigvecs_cupy[:, xi*Nx**2:(xi+1)*Nx**2, 0],
13                           eigvecs_cupy[:, xi*Nx**2:(xi+1)*Nx**2, 1],
14                           eigvecs_cupy[:, xi*Nx**2:(xi+1)*Nx**2, 2],
15                ) + ... # 5 more cyclic permutations with signs
16     return VVV_sink # shape: [Nt, Nev1, Nev1, Nev1]

```

VVV 的物理构造

VVV 的核心是一个三重量子力学波函数的颜色单态投影：

$$VVV^{(a,b,c)}(t; \vec{P}) = \sum_{\vec{x}} e^{-i\vec{P} \cdot \vec{x}} \varepsilon_{ijk} v_i^{(a)}(\vec{x}, t) v_j^{(b)}(\vec{x}, t) v_k^{(c)}(\vec{x}, t) \quad (21)$$

其中 ε_{ijk} 是 SU(3) 的完全反对称张量，确保三夸克组成颜色单态。代码中的六个 contract 项正好对应三夸克波函数的反对称化：

$$\text{项 1: } \varepsilon_{ijk} v_a^i v_b^j v_c^k = v_a^{\text{col0}} v_b^{\text{col1}} v_c^{\text{col2}} \quad (22)$$

$$\text{项 2: } \varepsilon_{ijk} v_b^i v_c^j v_a^k \quad (\text{轮换}) \quad (23)$$

$$\text{项 3: } \varepsilon_{ijk} v_c^i v_a^j v_b^k \quad (\text{轮换}) \quad (24)$$

$$\text{项 4-6: 负号项, 对应 } \varepsilon_{ikj} \text{ 型排列} \quad (25)$$

分块策略：由于 GPU 显存限制（中间张量过大），沿 x 方向分 N_x 块处理，每块依次计算后累加到结果中。这是典型的显存-计算折中方案。

5.5 Wick 收缩与 2pt 关联函数

质子 2pt 关联函数通过对 perambulator 和 VVV 的缩并得到：

Wick 收缩—2pt_proton_Cg5gm_u_L32x64_mom2_xdir_gpu.py:360-398

```

1 for t_source in range(0, Nt):
2     VVV_source = cp.conj(VVV_sink[t_source])
3     peram_u = readin_peram(peram_u_dir, conf_id, Nt, Nev1,
4         t_source)
5     # 宇称投影的 perambulator: Gamma5 * peram * interpolator
6     CG5peram_uCG5 = contract(
7         "gh,thkbe,jk->tgjbe",
8         interProject1_cupy, peram_u, interProject2_cupy
9     )
10    for t_sink in range(0, Nt):
11        deltat = (t_sink - t_source + Nt) % Nt
12        if 2 <= deltat <= 30:
13            contrac_nucl_matrix[t_sink, t_source] = (
14                contract(
15                    "abc,gjad,gjbe,ilcf,def->il",
16                    VVV_sink[t_sink],          # VVV(汇)
17                    peram_u[t_sink],           # peram(汇)
18                    CG5peram_uCG5[t_sink],     # Gamma5 * peram * 插值
19                    peram_u[t_sink],           # peram(汇)
20                    VVV_source,                # VVV(源)†
21                )
22            - contract( # 交换项 (费米子反对称性)
23                "abc,glaf,gjbe,ijcd,def->il",
24                VVV_sink[t_sink],
25                peram_u[t_sink],
26                CG5peram_uCG5[t_sink],
27                peram_u[t_sink],
28                VVV_source,
29            )

```

Wick 收缩的物理图像

质子 2pt 关联函数的收缩涉及六个张量的缩并 (如图??):

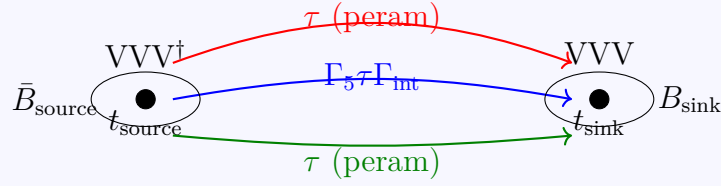


图: 质子 2pt 关联函数的 Wick 收缩示意图: 三个夸克传播子连接源和汇

两个 contract 项对应于两个 Wick 收缩 (直接项和交换项)。Dirac 空间缩并的结果是一个 4×4 矩阵 $[i, l]$:

$$C_2(t_{\text{sink}}, t_{\text{source}})_{il} = \langle B_i(t_{\text{sink}}) \bar{B}_l(t_{\text{source}}) \rangle \quad (26)$$

其中 B_i 是插值算符 (interpolator), 通常取为:

$$B_\alpha = \varepsilon_{abc} (u_a^T C \gamma_5 d_b) u_{c,\alpha} \quad (27)$$

5.6 宇称投影

强子关联函数需要投影到确定的宇称态。在 DeGrand-Rossi 基下:

宇称投影 (2pt script)

```

1 matrix_pplus = 0.5 * (gamma(0) + gamma(4)) # 正宇称: (1+
    gamma4)/2
2 matrix_pminus = 0.5 * (gamma(0) - gamma(4)) # 负宇称: (1-
    gamma4)/2
3
4 # 投影到正/负宇称
5 contrac_nucl_pp = contract("li,yxil->yx", matrix_pplus,
    contrac_nucl_matrix)
6 contrac_nucl_pm = contract("li,yxil->yx", matrix_pminus,
    contrac_nucl_matrix)
7
8 # 反周期边界条件的符号修正
9 for t_source in range(0, Nt):
10     for t_sink in range(0, Nt):
11         if t_sink < t_source:
12             contrac_nucl_pp[t_sink, t_source] *= -1.0
13         if t_sink > t_source:
14             contrac_nucl_pm[t_sink, t_source] *= -1.0

```

宇称投影算符

在欧几里得空间中，正/负宇称投影算符为：

$$\Gamma_{\pm} = \frac{1}{2}(1 \pm \gamma_4) \quad (28)$$

在 DeGrand-Rossi (DR) 手征基下， γ_4 是非对角的：

$$\gamma_4^{\text{DR}} = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} \quad (29)$$

因此 $\Gamma_+ = \frac{1}{2}(\mathbb{1} + \gamma_4)$ 筛选正宇称态， $\Gamma_- = \frac{1}{2}(\mathbb{1} - \gamma_4)$ 筛选负宇称态。

反周期边界条件：费米子场在时间方向满足反周期边界条件 $\psi(t + N_t) = -\psi(t)$ 。当 $t_{\text{sink}} < t_{\text{source}}$ 时（即关联函数“绕回”），需要乘上 -1 因子来正确处理边界的相位。

6 Gamma 矩阵：DeGrand-Rossi (DR) 基

6.1 DR 基的显式表示

donghx 代码使用 DeGrand-Rossi (DR) 手征基 [11] 表示 gamma 矩阵。DR 基是手征表示的一种变体，其中 γ_5 是对角的：

DR 基 Gamma 矩阵

$$\gamma_0^{\text{DR}} = \mathbb{1}_{4 \times 4} \qquad \gamma_1^{\text{DR}} = \begin{pmatrix} 0 & 0 & 0 & i \\ 0 & 0 & i & 0 \\ 0 & -i & 0 & 0 \\ -i & 0 & 0 & 0 \end{pmatrix} \qquad (30)$$

$$\gamma_2^{\text{DR}} = \begin{pmatrix} 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \end{pmatrix} \qquad \gamma_3^{\text{DR}} = \begin{pmatrix} 0 & 0 & i & 0 \\ 0 & 0 & 0 & -i \\ -i & 0 & 0 & 0 \\ 0 & i & 0 & 0 \end{pmatrix} \qquad (31)$$

$$\gamma_4^{\text{DR}} = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} \qquad \gamma_5^{\text{DR}} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix} \qquad (32)$$

6.2 代码实现

Gamma 矩阵定义—gamma_matrix_cupy_DR.py:6-109

```

1 import cupy as cp
2
3 g0 = cp.eye(4, dtype=complex) # identity
4
5 g1 = cp.zeros((4,4), dtype=complex)
6 g1[0,3] = 1j; g1[1,2] = 1j; g1[2,1] = -1j; g1[3,0] = -1j
7
8 g2 = cp.zeros((4,4), dtype=complex)
9 g2[0,3] = -1; g2[1,2] = 1; g2[2,1] = 1; g2[3,0] = -1
10
11 g3 = cp.zeros((4,4), dtype=complex)
12 g3[0,2] = 1j; g3[1,3] = -1j; g3[2,0] = -1j; g3[3,1] = 1j
13
14 g4 = cp.zeros((4,4), dtype=complex)
15 g4[0,2] = 1; g4[1,3] = 1; g4[2,0] = 1; g4[3,1] = 1
16
17 g5 = cp.zeros((4,4), dtype=complex)
18 g5[0,0] = 1; g5[1,1] = 1; g5[2,2] = -1; g5[3,3] = -1

```

6.3 插值算符 (Interpolator) 的构造

插值算符是 gamma 矩阵的组合，用于构造具有特定量子数的强子算符：

插值算符构造—gamma_matrix_cupy_DR.py:49-106

```

1 def gamma(i):
2     if i == 0: return g0                # identity
3     elif i == 1: return g1              # gamma1
4     elif i == 2: return g2              # gamma2
5     elif i == 3: return g3              # gamma3
6     elif i == 4: return g4              # gamma4
7     elif i == 5: return g5              # gamma5
8     elif i == 6: return cp.matmul(g2, g3) # -gamma1*
        gamma4*gamma5
9     elif i == 7: return cp.matmul(g3, g1) # -gamma2*
        gamma4*gamma5
10    elif i == 8: return cp.matmul(g1, g2) # -gamma3*
        gamma4*gamma5
11    elif i == 9: return cp.matmul(g1, g4) # gamma1*
        gamma4
12    elif i == 10: return cp.matmul(g2, g4) # gamma2*
        gamma4
13    elif i == 11: return cp.matmul(g3, g4) # gamma3*
        gamma4
14    elif i == 12: return cp.matmul(g1, g5) # gamma1*
        gamma5
15    elif i == 13: return cp.matmul(g2, g5) # gamma2*
        gamma5
16    elif i == 14: return cp.matmul(g3, g5) # gamma3*
        gamma5
17    elif i == 15: return cp.matmul(g4, g5) # gamma4*
        gamma5
18    elif i == 16:                        # (gamma3*
        gamma1)*(1+gamma4)/2
        return cp.matmul(cp.matmul(g3,g1), 0.5*(g0+g4))
19
20    elif i == 17:                        # (gamma3*
        gamma1)*(1-gamma4)/2
        return cp.matmul(cp.matmul(g3,g1), 0.5*(g0-g4))
21

```

插值算符与质子量子数

在 2pt 代码中，关键的插值算符组合包括 (element="_Cg5g4" 时)：

$$\Gamma_{\text{proj}} = \gamma_7 = -\gamma_2\gamma_4\gamma_5 = \gamma_3\gamma_1 \quad (33)$$

这对应于 $\gamma_3\gamma_1$ 型插值算符，用于筛选质子的特定自旋态。插值算符的完整作用形式为：

$$\tau \rightarrow \Gamma_{\text{proj}} \cdot \tau \cdot \Gamma_{\text{proj}} \quad (34)$$

即对 perambulator 的 Dirac 指标同时从左右两侧进行投影。

7 GPU 计算策略与数据管理

7.1 CuPy 后端

donghx 代码主要使用 CuPy 作为 GPU 计算后端 (_gpu.py 文件)。CuPy 提供了与 NumPy 兼容的 API，但在 GPU 上执行：

GPU 数据传输—多处示例

```

1 import cupy as cp
2
3 # CPU -> GPU: 将 numpy 数组传到 GPU
4 eigvecs_cupy = cp.asarray(Eigvec)
5 peram_cupy = cp.array(peram)
6
7 # GPU -> CPU: 将结果取回 CPU
8 result_np = cp.asnumpy(gpu_result)
9
10 # GPU 张量缩并
11 result = contract("abc,ade,bdf,cef->", A, B, C, D)
12
13 # GPU 内存管理
14 cp._default_memory_pool.free_all_blocks()
15 cp.cuda.Device().synchronize()

```

7.2 内存管理策略

面对大规模格点数据，代码采用了多种内存管理策略：

- (1) **按需读入:** perambulator 按 t_{source} 逐个读入, 而非一次性加载全部时间片。这大幅减少了 GPU 显存占用 (典型情况: $N_t = 64$ 时减少约 64 倍)。
- (2) **空间分块 (Tiling):** VVV 计算中将 x 方向分成 N_x 块, 逐块缩并后累加。这避免了中间张量 $\mathcal{O}(N_{\text{ev}}^3 \times N_x^3)$ 的巨大内存占用。
- (3) **显式内存释放:** 使用 `cp._default_memory_pool.free_all_blocks()` 在每个 t_{source} 循环后释放 GPU 内存。
- (4) **预计算与缓存:** Plaquette 可以预先计算并存储为 .npz 文件 (`Calc_pla.py`), 后续 OPE 算符计算直接加载 (`Calc_ope_unpol_new.py`), 避免重复计算。

7.3 opt_einsum 加速

代码中使用 `opt_einsum.contract` 替代 NumPy/CuPy 原生的 `einsum`, 利用优化的缩并路径减少计算量:

opt_einsum 使用

```

1 from opt_einsum import contract
2
3 # 自动寻找最优缩并路径的 einsum
4 result = contract("abc,gjad,gjbe,ilcf,def->il",
5                   VVV_sink[t_sink], peram_u[t_sink],
6                   CG5peram_uCG5[t_sink], peram_u[t_sink],
7                   VVV_source)
```

对于复杂的多张量缩并 (如质子 2pt 中的五张量缩并), `opt_einsum` 通过动态规划找到计算量最小的缩并顺序, 通常能将计算复杂度降低一个量级。

7.4 DCU (AMD/Hygon) 后端

以 `_dcu.py` 结尾的文件使用 DCU 后端 (AMD ROCm/HIP), 通过 PyTorch 兼容层实现 GPU 计算。这些文件的代码结构与 GPU 版本相似, 但使用 PyTorch 张量代替 CuPy。

表 2: donghx 代码中的文件 I/O 格式汇总

数据类型	文件格式	读入方法	示例文件
规范组态	ILDG 二进制	<code>np.fromfile</code>	<code>...ildg-binary-data</code>
本征矢量	原始二进制 (f8)	<code>np.fromfile</code>	<code>eigvecs_t000_20000</code>
Perambulator	原始二进制 (f8)	<code>np.fromfile</code>	<code>perams.20000.0.0</code>
VVV	原始二进制 (f8)	<code>np.fromfile</code>	<code>VVV.t000.Px0Py0Pz3.conf20000</code>
VdV	原始二进制 (f8)	<code>np.fromfile</code>	<code>VdaggerV.Px0Py0Pz3.conf20000</code>
Plaquette	NumPy npz	<code>np.load</code>	<code>ops_pla_conf20000.npz</code>
OPE 结果	NumPy npy	<code>np.save/np.load</code>	<code>ops_mu3_nu0_dz15_conf20000.npy</code>
2pt 结果	NumPy npy	<code>cp.save/np.load</code>	<code>twopt_slice_pp_Px3Py0Pz0_...npy</code>
2pt 结果 (ASCII)	文本 dat	<code>write_data_ascii</code>	<code>corr_Nucleon_pp_...dat</code>

7.5 文件 I/O 与数据格式

8 完整计算流程与数据流

8.1 端到端流程

图??展示了从规范组态到胶子 PDF 的完整计算数据流：

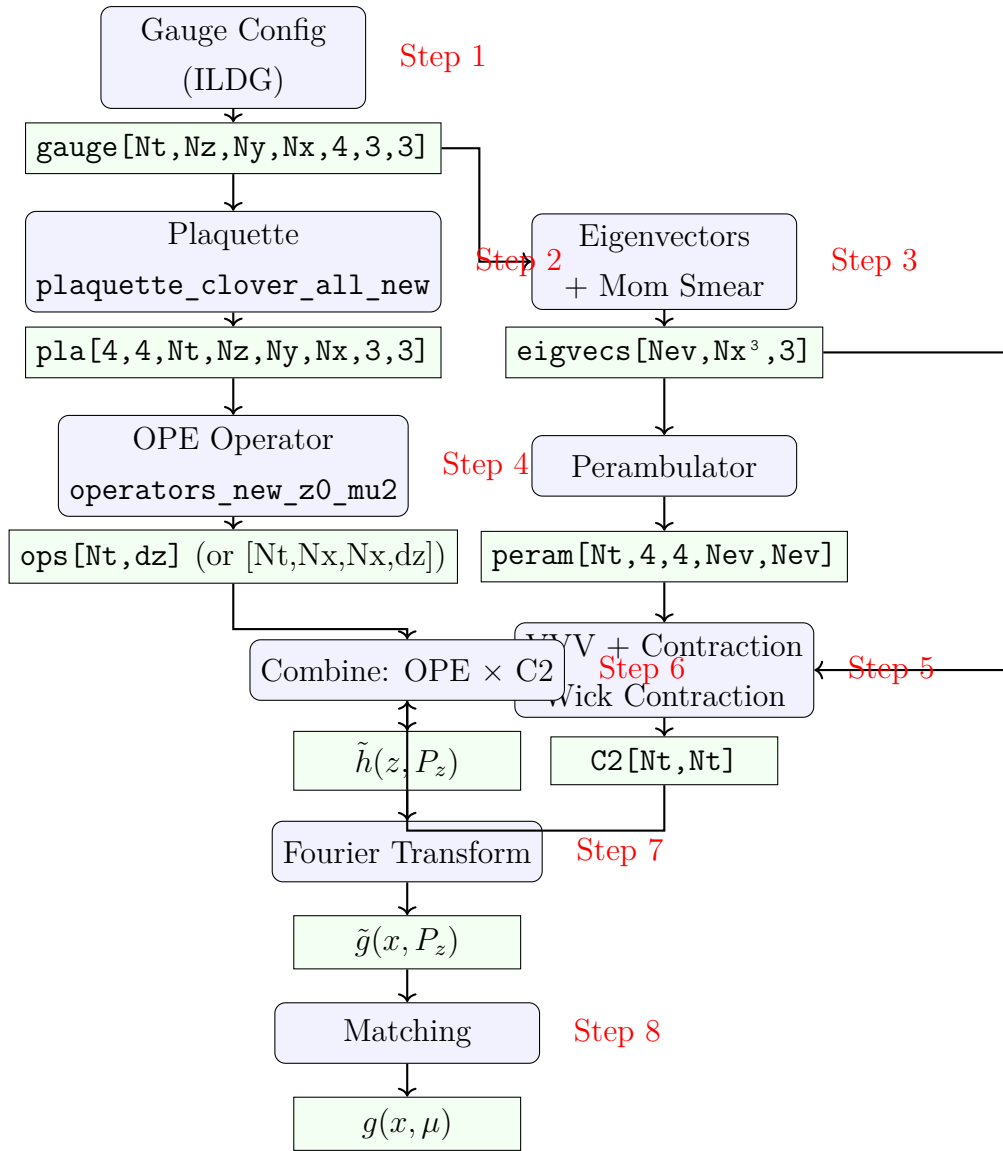


图: 胶子 PDF 计算的完整数据流图: 8 个步骤从规范组态到光锥 PDF

表 3: 计算步骤与代码文件对应表

步骤	物理内容	主要代码文件
1	规范场读入	Calc_pla.py, Calc_ope_unpol.py
2	Plaquette/ $F_{\mu\nu}$ 构造	Operator.py (plaquette_clover_*)
3	本征矢量 + 动量涂抹	2pt_proton_Cg5gmu*_gpu.py
4	OPE 非定域算符	Calc_ope_unpol.py, Calc_ope_helicity.py
5	VVV+Wick 收缩	Calc_VVV.py, 2pt_proton_Cg5gmu*_gpu.py
6	组合: OPE $\times$ C2	zhangxin 工作流 (gluon_pdf_workflow.py)
7	Fourier 变换	zhangxin 分析代码 (main.py)
8	微扰匹配	LaMET 匹配核代码

8.2 各步骤的代码对应

8.3 典型运行命令

典型运行示例

Plaquette 预计算:

```
1 python Calc_pla.py < pla_params.txt
2 # pla_params.txt:
3 #   Nt 64 / Nx 32 / conf_id 20000
4 #   conf_file /path/to/config.dat
5 #   pla_dir /path/to/output/
```

OPE 算符计算 (3-rank MPI):

```
1 mpirun -np 3 python Calc_ope_unpol.py < ope_params.txt
2 # ope_params.txt:
3 #   Nt 64 / Nx 32 / delta_z 15 / conf_id 20000
4 #   conf_file /path/to/config.dat
5 #   link_dir z
6 #   output_dir /path/to/ope_output/
```

质子 2pt 关联函数 (GPU 单卡):

```
1 python 2pt_proton_Cg5gmu_L32x64_mom2_xdir_gpu.py 20000
2 # 硬编码参数: Nt=64, Nx=32, Nev=100, Pz=0, mom_smeas=3
3 # 关键输出: twopt_slice_pp-Px3Py0Pz0_...npy
```

9 总结

本文档系统地分析了 donghx 在 `/root/lattice-pdf/examples/donghx/` 中的全部格点 QCD 代码，建立了物理公式与 GPU 代码实现之间的一一对应关系。主要涵盖以下内容：

1. **理论框架**: LaMET 方法将格点上的准 PDF 与光锥 PDF 通过微扰匹配联系起来。胶子 PDF 涉及非连通图，三点关联函数被分解为质子 2pt 部分和 OPE 胶子算符部分。
2. **场强张量构造**: 格点上的 $F_{\mu\nu}$ 通过 Clover (四叶草) 改进的 plaquette 构造，四个定向 plaquette 的对称组合将离散化误差降至 $\mathcal{O}(a^2)$ 。代码中通过 `plaquette_clover_all_new` 和 `plaquette_clover_all_tilde` 分别计算 $F_{\mu\nu}$ 和其对偶 $\tilde{F}_{\mu\nu}$ 。
3. **非定域 OPE 算符**: 通过 Wilson 线连接两个位置的场强张量。`operators_new_z0_mu2` 等函数实现了各种空间构型 (正向/反向 Wilson 线、不同 Lorentz 指标组合)。MPI 并行将不同 (μ, ν) 分量分配给不同进程。
4. **质子 2pt 关联函数**: 使用 distillation 方法, 通过低模拉普拉斯本征矢量、perambulator 和 VVV 三重量子数进行 Wick 收缩。动量涂抹通过注入平面波相位 $\exp(-i\vec{\zeta} \cdot \vec{x})$ 提高高动量态 overlap。
5. **GPU 优化**: CuPy 后端、`opt_einsum` 缩并优化、按需读入、空间分块和显式内存管理等策略保证了在大规模格点上的计算效率。

本分析覆盖了 donghx 目录中的 68 个文件, 包括核心模块 (`Operator.py`, `gamma_matrix_cupy_DR`, `input_output_4_cupy.py`)、OPE 计算脚本 (`Calc_ope_unpol.py`, `Calc_ope_helicity.py`, `Calc_ope_gauge_fix_unpol.py`, `Calc_ope_helicity_new.py`, `Calc_ope_unpol_new.py`, `Calc_ope_helicity_test.py`, `Calc_ope_gauge_fix_helicity.py`)、plaquette 预计算 (`Calc_pla.py`)、VVV 计算 (`Calc_VVV.py`)、质子 2pt 脚本 (40+ 个变体, 覆盖不同格点尺寸、动量和 GPU/DCU 后端) 以及数据检查工具 (`check_exist.py`, `check_exist_TMD.py`)。

参考文献

- [1] X. Ji, “Parton Physics on a Euclidean Lattice,” *Phys. Rev. Lett.* **110**, 262002 (2013) [[arXiv:1305.1539](#)].
- [2] X. Ji, “Parton Physics from Large-Momentum Effective Field Theory,” *Sci. China Phys. Mech. Astron.* **57**, 1407-1412 (2014) [[arXiv:1404.6680](#)].
- [3] X. Ji, Y.-S. Liu, Y. Liu, J.-H. Zhang, and Y. Zhao, “Large-Momentum Effective Theory,” *Rev. Mod. Phys.* **93**, no.3, 035005 (2021) [[arXiv:2004.03543](#)].

- [4] 内部理论笔记, “Note of gluon PDFs,” `/root/lattice-pdf/docs/Note of gluon PDFs.pdf`.
- [5] T. Khan *et al.* (HadStruc Collaboration), “First Nucleon Gluon PDF from Large Momentum Effective Theory,” [[arXiv preprint](#)]. (见 `/root/lattice-pdf/docs/First Nucleon Gluon PDF from Large Momentum Effective Theory.pdf`)
- [6] W. Wang, J.-H. Zhang, S. Zhao, and R. Zhu, “Gluon Quasi-PDF From Lattice QCD,” *Phys. Rev. D* **100**, 074509 (2019). (见 `/root/lattice-pdf/docs/Gluon Quasi-PDF From Lattice QCD.pdf`)
- [7] C. Egerer *et al.* (HadStruc Collaboration), “Distillation at High-Momentum,” *Phys. Rev. D* **103**, 034502 (2021) [[arXiv:2009.10891](#)]. (见 `/root/lattice-pdf/docs/Distillation at High-Momentum.pdf`)
- [8] M. Peardon *et al.* (Hadron Spectrum Collaboration), “A Novel Quark-Field Creation Operator Construction for Hadronic Physics in Lattice QCD,” *Phys. Rev. D* **80**, 054506 (2009) [[arXiv:0905.2160](#)].
- [9] G. Bali, B. Lang, B. Lucini, B. Lüscher, and S. Romiti, “Kinematically-Enhanced Interpolating Operators for Boosted Hadrons,” (见 `/root/lattice-pdf/docs/Kinematically-enhanced interpolating operators for boosted hadrons.pdf`)
- [10] G. Bali *et al.*, “Momentum-Smeared Interpolating Operators,” (2016).
- [11] T. DeGrand and P. Rossi, “Conditioning Techniques for Dynamical Fermions,” *Comput. Phys. Commun.* **60**, 211 (1990).
- [12] C. Gattringer and C. B. Lang, *Quantum Chromodynamics on the Lattice*, Springer (2010). (见 `/root/lattice-pdf/books/Quantum Chromodynamics on the Lattice.pdf`)
- [13] X. Ji, Y. Liu, A. Schäfer, W. Wang, Y.-B. Yang, J.-H. Zhang, and Y. Zhao, “A Hybrid Renormalization Scheme for Quasi Light-Front Correlations in Large-Momentum Effective Theory,” *Nucl. Phys. B* **964**, 115311 (2021). (见 `/root/lattice-pdf/docs/A Hybrid Renormalization Scheme for Quasi Light-Front Correlations in Large-Momentum Effective Theory.pdf`)
- [14] “First Self-Renormalized Gluon PDF of Nucleon from Large-Momentum Effective Theory in the Continuum Limit,” (见 `/root/lattice-pdf/docs/First Self-Renormalized Gluon PDF of Nucleon from Large-Momentum Effective Theory in the Continuum Limit.pdf`)

- [15] Y. Li *et al.*, “Impact of gauge fixing precision on the continuum limit of non-local quark-bilinear lattice operators,” (见 `/root/lattice-pdf/docs/Impact of gauge fixing precision on the continuum limit of non-local quark-bilinear lattice operators.pdf`)
- [16] “Leading Power Accuracy in Lattice Calculations of Parton Distributions,” (见 `/root/lattice-pdf/docs/Leading Power Accuracy in Lattice Calculations of Parton Distributions.pdf`)
- [17] B. Sheikholeslami and R. Wohlert, “Improved Continuum Limit Lattice Action for QCD with Wilson Fermions,” Nucl. Phys. B **259**, 572 (1985).
- [18] M. Lüscher, “Properties and uses of the Wilson flow in lattice QCD,” JHEP **08**, 071 (2010) [[arXiv:1006.4518](#)].
- [19] “Gluon Parton Distribution of the Nucleon from 2+1+1-Flavor Lattice QCD in the Physical-Continuum Limit,” (见 `/root/lattice-pdf/docs/Gluon Parton Distribution of the Nucleon from 2+1+1-Flavor Lattice QCD in the Physical-Continuum Limit.pdf`)